

Extended Methods Appendix: Data Cleaning Procedures for Housing Discrimination Study (2012) Analysis

1 Overview

This appendix provides comprehensive documentation of the data cleaning procedures applied to the Housing Discrimination Study (HDS) 2012 dataset. The cleaning process transforms raw SAS data files into analysis-ready datasets suitable for econometric analysis.

2 Data Sources and Initial Processing

2.1 Raw Data Structure

The HDS 2012 data consists of six primary SAS files in `.sas7bdat` format. Each file serves a distinct purpose in documenting the field experiment:

The **assignment** file links individual testers to specific test assignments and contains release status indicators. The **taf** (Test Assignment Form) file provides test-level characteristics and parameters. The **sales** file contains detailed records of each sales test visit, including appointment times and outcomes. The **tester_censored** file provides tester demographic and background information with personally identifiable information removed. The **rhgeo** file contains geocoded data for recommended homes shown during each test. Finally, the **rechomes** file provides additional information about recommended properties.

2.2 Sales Test Filtering

The HDS 2012 study encompasses both rental and sales discrimination tests. For this analysis, we focus exclusively on sales tests, which are identified through control numbers containing the pattern “-S[A-Z]-”. This pattern corresponds to three test types: SA for Asian testers, SB for Black testers, and SH for Hispanic testers.

The filtering process is applied consistently across all datasets:

```
1 assignment <- assignment_raw %>%  
2   filter(grepl("-S[A-Z]-", CONTROL)) %>%  
3   filter(RELEASE == "1" & !is.na(TESTERID))
```

```

4
5 taf <- taf_raw %>%
6   filter(grepl("-S[A-Z]-", CONTROL))

```

For the assignment file, we apply an additional filter requiring `RELEASE = "1"`, which indicates completed and valid test assignments. This ensures our analysis includes only tests that were successfully executed in the field.

3 Date and Time Processing

3.1 The Challenge of Date Format Heterogeneity

One of the most significant data cleaning challenges involves parsing appointment dates (SAPPTD) and birth dates (TDOB), which exhibit remarkable format diversity. Our analysis identified over 15 distinct date format patterns in the raw data, reflecting the challenges of manual data entry across many distinct testers.

The date formats encountered include standard formats with various separators (“MM/DD/YYYY”, “MM-DD-YYYY”, “MM.DD.YYYY”), formats without separators (“MMDDYYYY”, “MMDDYY”), ambiguous formats that could represent multiple date interpretations (“MM/YY”, “DD/MM”, “MMDD/YY”), and numerous data entry errors including impossible dates (“6/31/12”), leap year errors (“02/29/2011”), and clear typographical errors (“08/18/7977”, “01/08/1064”).

3.2 Date Parsing Algorithm

To address this heterogeneity, we developed a comprehensive parsing function that employs multiple strategies. First, the function standardizes all separators (dashes, dots, spaces) to forward slashes and removes extraneous characters. Second, it maintains a lookup table for known invalid dates that require manual correction. Third, it uses regular expression pattern matching to identify date formats, testing patterns in order of specificity. Fourth, it handles century ambiguity through context-aware logic: for birth dates, two-digit years are interpreted as 19XX (reflecting that testers were born in the 1900s), while for appointment dates, two-digit years are interpreted as 20XX (reflecting that tests occurred in 2011-2012).

The date parsing function handles ambiguous two-number date formats, such as “MM/YY” or “DD/MM” by first checking if the first number is greater than 12, which would make it invalid as a month. In this case, it assumes the date is in a “DD/MM” format and the year is inferred later in the data cleaning process.

If both numbers could plausibly represent either a month (1–12) or a day (1–31), the function uses additional context to resolve the ambiguity. For birth dates, it first attempts to interpret the date as “MM/YY” and calculates the resulting age as of 2011. If this age is reasonable—between 18 and 90 years old—it accepts this interpretation. For appointment dates, the function checks

if the second number is 11 or 12, which likely indicates the years 2011 or 2012. If so, it interprets the date as "MM/YY". If not, it defaults to interpreting the date as "DD/MM" with the placeholder year 2000, again to be corrected later.

Finally, if the first number is too large to represent a valid month or day (greater than 31), the function considers the possibility of a "YY/MM" format. For birth dates, it again validates the resulting age, ensuring it falls within a plausible range (18–90 years old in 2011). For appointment dates, it specifically checks if the first number is 11 or 12, indicating the years 2011 or 2012, and interprets the date accordingly.

If none of these logical steps yield a valid interpretation, the function returns a missing value (NA) and issues a warning. This structured and context-aware approach ensures ambiguous date formats are handled consistently, transparently, and logically, with clear reasoning guiding each decision.

```

1 # Example of ambiguous format handling
2 "~\\d{1,2}/\\d{2}$" = function(x) {
3   parts <- strsplit(x, "/")[1]
4   first_num <- as.numeric(parts[1])
5   second_num <- as.numeric(parts[2])
6
7   could_be_month <- function(n) !is.na(n) && n >= 1 && n <= 12
8   could_be_day <- function(n) !is.na(n) && n >= 1 && n <= 31
9
10  # If first number too large for month, likely DD/MM format
11  if (!could_be_month(first_num) && could_be_day(first_num) &&
12      could_be_month(second_num)) {
13    if (!is_birthday_date) {
14      return(as.Date(sprintf("%02d/%02d/2011", second_num, first_
15        num),
16                      format = "%m/%d/%Y"))
17    }
18  }
19  # Additional logic for other scenarios...
20 }
```

3.3 Systematic Date Correction

Some parsed appointment dates fall outside the study period (July 2011 through October 2012), indicating data entry errors where incorrect years or months were recorded. Other times, years are missing. Our correction algorithm addresses these systematically by leveraging the fact that multiple appointments within the same test (identified by **CONTROL** number) should occur within a reasonable timeframe.

The correction process identifies all dates outside the valid study period, then for each invalid date, searches for other appointments with the same **CONTROL** number that have valid dates. When valid reference dates exist, the algorithm preserves the day information from the invalid date but adopts the year from the valid appointments. If the corrected year matches the original (suggesting a month error rather than a year error), the algorithm adjusts the month instead.

When no valid reference date exists for a given `CONTROL`, the appointment date is set to missing rather than imputed.

3.4 Time Parsing and Correction

Appointment times (`SBEGH`) present similar challenges, with formats including military time (“0315”, “1430”), three-digit formats (“357” interpreted as 3:57), various separated formats (“3:15”, “3;15”, “3.15”), and invalid entries (“N/A”, non-numeric values). The complexity is compounded by AM/PM indicators (`SBEGAM`) that are sometimes missing or clearly incorrect.

Our time parsing function handles these variations systematically. For missing AM/PM indicators, the function infers the most likely period based on the hour: times between 9 and 12 are likely morning appointments, while times between 1 and 8 are likely afternoon or evening appointments. The function also corrects obviously incorrect indicators, such as times between 1:00 and 5:00 marked as AM (changed to PM) or times between 10:00 and 11:00 marked as PM (changed to AM).

A particular challenge involves appointments recorded at exactly midnight (00:00:00), which is implausible for real estate showings. Our analysis revealed that these entries typically represent data entry errors where time information was missing. To address this, we identify all midnight appointments and check whether other appointments exist on the same day for the same `CONTROL`. If multiple appointments exist on the same day, the midnight time is set to missing, indicating a time entry error. Midnight times are preserved only when they represent the sole appointment for that day, though even these cases likely represent missing data. This is done because appointment times mainly enter the analysis as covariates for visit order, and inaccurate timing does not impact visit order when no other appointments exist for that day.

4 Data Aggregation and Variable Construction

4.1 Visit Order Determination

The experimental design allowed testers to make multiple visits to the same real estate agent. To properly analyze outcomes, we must establish the chronological order of these visits. Our approach groups records by `CONTROL` number, sorts by the combined appointment date and time, and assigns visit order using sequential numbering. Records with missing datetime information are placed last in the sequence.

4.2 Creating the Analysis Dataset

The final sales dataset aggregates information across multiple visits for each tester-control combination. This aggregation preserves both visit-specific information from the first interaction and cumulative information across all visits. The aggregation process creates several categories of variables:

First visit metrics capture the initial interaction between tester and agent, including `SBEGAM_FIRST` (appointment time AM/PM indicator), `STOTUNIT_FIRST` (number of units shown), and `SAVLBAD_FIRST` (indicator for whether the tester was told the house in the ad was available). Aggregate metrics summarize the entire test sequence, including `num_visits` (total number of visits), `STOTUNIT_TOTAL` (total units shown across all visits), and `SAVLBAD_ANY` (indicator for whether the original ad house was indicated to be available in any visit). Temporal metrics provide timing information, including `first_visit_datetime` and `was_first_visitor` (indicator for whether this tester was the first to visit the agent in this test).

The aggregation also includes extraction of race categories from the `RACEID` variable. This extraction uses regular expressions to identify the numeric race code embedded at the end of the race identifier string. Additionally, the process includes special handling for infinite values that can arise when calculating minimum datetimes across empty sets, converting these to proper missing values.

```

1 sales_final <- sales_with_time %>%
2   group_by(CONTROL) %>%
3   arrange(appointment_datetime) %>%
4   mutate(visit_order = row_number()) %>%
5   ungroup() %>%
6   group_by(CONTROL, TESTERID) %>%
7   summarise(
8     RACEID = first(RACEID),
9     SBEGAM_FIRST = first(SBEGAM[which.min(visit_order)]),
10    STOTUNIT_FIRST = first(STOTUNIT[which.min(visit_order)]),
11    SAVLBAD_FIRST = first(SAVLBAD_BINARY[which.min(visit_order)]),
12    first_visit_datetime = min(appointment_datetime, na.rm = TRUE),
13    num_visits = n(),
14    STOTUNIT_TOTAL = sum(STOTUNIT, na.rm = TRUE),
15    SAVLBAD_ANY = as.numeric(any(SAVLBAD_BINARY == 1, na.rm = TRUE)),
16    was_first_visitor = any(visit_order == 1, na.rm = TRUE),
17    .groups = 'drop'
18  ) %>%
19  mutate(
20    RACE = as.numeric(str_extract(as.character(RACEID), "\\d$")),
21    first_visit_datetime = if_else(is.infinite(first_visit_datetime),
22                                   as.POSIXct(NA), first_visit_
23    datetime)
24  )

```

Note: The only occurrence where `appointment_datetime` is missing while `appointment_date` is present represents a data entry correction. Specifically, this case occurs when an initial record contained an invalid time entry (typically 00:00), prompting data entry personnel to create a subsequent record with the corrected time information. The original record with the invalid time is retained in the dataset but lacks a valid `appointment_datetime`. This incomplete record can be and is safely excluded from analysis as it represents a superseded data entry rather than genuine missing information.

5 Tester Demographics Processing

Tester birth dates undergo the same parsing process as appointment dates, with the `is_birth_date` parameter set to TRUE to apply appropriate century logic. Ages are calculated as of January 1, 2012, representing the approximate midpoint of the study period. The calculation uses precise day-based computation (days divided by 365.25 to account for leap years), rounds to the nearest whole year, and excludes implausible ages (under 18 years old) by setting them to missing.

The final step merges the aggregated sales data with tester demographics using left joins on `TESTERID`, preserving all sales records even when tester demographic information may be incomplete.

6 Output Files

The cleaning process generates two primary output files. First, `sales_cleaned.csv` contains the aggregated sales data with one row per tester-control combination, suitable for most analyses of discrimination outcomes. Second, `sales_and_tester_merged.csv` includes the sales data merged with tester demographics, enabling analyses that incorporate tester characteristics.

7 Recommended Homes Data Integration

7.1 Data Deduplication and Quality Control

7.1.1 Rechomes Data Deduplication

The recommended homes (`rechomes`) dataset contains records of properties shown to testers during their visits. Initial inspection revealed the presence of duplicate records that needed systematic resolution before analysis. The deduplication process addresses duplicates arising from multiple sources, including data entry errors and repeated records of the same property.

Our deduplication strategy identifies duplicate records based on the complete address specification: `CONTROL`, `TESTERID`, `HSITEAD` (street address), `HSITETYP` (site type), `HUNITNO` (unit number), `HCITY`, `HSTATE`, and `HZIP`. When duplicates are identified, we retain the first occurrence and remove subsequent records. This approach assumes that the first entry represents the original data capture, with subsequent duplicates arising from data processing or entry errors.

The deduplication process reduced the rechomes dataset from 24,859 initial rows to 24,679 unique property-tester combinations, removing 180 duplicate records (0.72% of the initial dataset). This modest reduction suggests generally good data quality with limited duplication issues.

7.1.2 Rhgeo Data Processing and Cleaning

The **rhgeo** dataset provides geocoded information for recommended homes, enabling geographic analysis of property recommendations. The processing of this dataset involves three sequential steps to ensure data quality and consistency.

First, we remove exact duplicate records that may have arisen during data processing or merging. This step reduced the dataset from 16,643 initial rows to 16,358 rows, removing 285 exact duplicates (1.71% of records). These duplicates likely arose from data processing errors rather than genuine repeated property showings.

Second, we identify and remove records with malformed or missing address information that would prevent meaningful geographic analysis. The filtering criteria exclude records where the street address field contains non-informative values such as “unknown”, “did not provide”, “MODEL HOME”, or variations thereof. We also exclude records with effectively empty addresses (containing only whitespace, dashes, or zeros) and addresses too short to be meaningful (three characters or fewer after trimming). This filtering removed 15 additional records with problematic addresses including: “255”, “Covington/Model Home”, “did not provide”, “MODEL HOME”, “Model home/ Castleberry model”, “Model home/Brookland model”, “Model home/Chelsea model”, “none”, “Sananah/Model Home”, “unknown”, and “unknown (Rangler house)”.

Third, we perform address-based deduplication similar to that applied to the **rechomes** data, retaining the first occurrence when multiple records share identical address components. This final step removed only one additional duplicate, yielding 16,342 unique geocoded property records.

7.1.3 Rechomes-Rhgeo Integration Analysis

Our merge analysis reveals an asymmetric relationship between these datasets. All 16,342 records in the deduplicated **rhgeo** dataset have corresponding entries in **rechomes**, indicating complete coverage of geocoded properties. However, 8,337 records in **rechomes** (33.78%) lack corresponding geocoded information in **rhgeo**.

Notably, the proportion of **rechomes** records with geocoding (66.22%) represents almost exactly two-thirds of the dataset, suggesting a systematic sampling or processing decision by the original data providers rather than geocoding failures. Investigation of the addresses that were and were not geocoded reveals no apparent systematic differences in address quality or format between the two groups. This pattern indicates that the original researchers likely geocoded a representative subset of properties rather than attempting comprehensive geocoding, possibly due to resource constraints or analytical requirements that did not necessitate complete geographic coverage.

The merged **rechomes_rhgeo** dataset retains all 24,679 records from the deduplicated **rechomes** data, with geocoded information available for the two-thirds subset that was processed in the original study. This approach preserves maximum information while clearly identifying which properties have

geographic data available for spatial analyses.

7.2 Analytical Sample Definition

Before merging recommended homes data, we first establish our analytical sample by applying key filtering criteria to ensure data quality and experimental validity. The analytical sample is constructed through a two-stage filtering process that addresses both outcome availability and experimental design requirements.

First, we exclude observations with missing outcome variables. Specifically, we require non-missing values for both `STOTUNIT_TOTAL` (total units shown across all visits) and `SAVLBAD_ANY` (whether the advertised property was available in any visit). These variables represent our primary measures of treatment by real estate agents and are essential for discrimination analysis.

Second, we enforce the paired testing experimental design by retaining only tests (`CONTROL` numbers) that include exactly two testers. This restriction ensures that each test includes one minority tester and one white tester, enabling direct pairwise comparisons that form the foundation of the experimental methodology.

```
1 analytic_sales_tester <- sales_and_tester %>%
2   filter(!is.na(STOTUNIT_TOTAL), !is.na(SAVLBAD_ANY)) %>%
3   group_by(CONTROL) %>%
4   filter(n() == 2) %>% # Exactly 2 testers per control
5   ungroup()
```

This filtering process demonstrates the data attrition inherent in creating a high-quality analytical dataset. From an initial 12,030 observations in the merged sales and tester data, requiring non-missing outcomes reduces the sample to 8,662 observations. Subsequently, requiring exactly two testers per control yields 7,302 observations representing 3,651 unique test pairs. The loss of observations primarily reflects incomplete data collection for some tests and instances where one tester in a pair failed to complete their assignment or had their data excluded during quality control.

7.3 Recommended Homes Data Merging

The merging process employs a left join on `CONTROL` and `TESTERID`, preserving all observations from the analytical sample while adding available property information from the merged `rechomes_rhgeo` dataset. This approach acknowledges that recommended homes data may not exist for all testers, either because they were shown no properties or because property data was not successfully collected or recorded.

```
1 sales_tester_rechomes <- analytic_sales_tester %>%
2   left_join(rechomes_rhgeo, by = c("CONTROL", "TESTERID")) %>%
3   mutate(has_rechomes_data = rowSums(!is.na(select(., HSITEAD, HZIP
4     ))) > 0)
```


7.3.1 Initial Merge Statistics

The merge with recommended homes data reveals several important patterns in data completeness. The analytical sample contains 3,651 unique test pairs (7,302 individual tester observations). Among these, 1,012 test pairs (27.7%) have at least one tester for whom recommended homes data is missing.

Examining the relationship between recommendations and data availability, we find that 2,717 test pairs (74.4%) involve cases where both testers received at least one property recommendation (i.e., neither tester has `STOTUNIT_TOTAL` = 0). Despite agents recommending properties, 158 individual observations (2.2% of the sample) show positive values for `STOTUNIT_TOTAL` but lack the corresponding detailed property information in the `rechomes` dataset. At the test-pair level, 143 controls (3.9%) have at least one tester who was shown properties but for whom detailed property data is missing.

7.3.2 Data Recovery Through Donor Imputation

A critical subset of the missing data represents cases where testers reported being shown only the advertised home (`STOTUNIT_TOTAL` = 1 and `SAVLBAD_ANY` = 1) but lack corresponding property details. Investigation revealed that in many such cases, the partnered tester within the same test had recorded information about the advertised property.

Our donor imputation approach leverages this within-test information:

1. Identify observations with `STOTUNIT_TOTAL` = 1, `SAVLBAD_ANY` = 1, and missing `rechomes` data
2. For each such observation, search for records within the same `CONTROL` where `HADHOME` = 1 (indicating the advertised home was recorded)
3. When a donor record is found, copy the property information (excluding `TESTERID`) to the observation with missing data

```
1 # Find potential donor rows (same CONTROL, HADHOME == 1)
2 donor_rows <- rechomes %>%
3   filter(CONTROL %in% missing_data$CONTROL, HADHOME == 1) %>%
4   select(-TESTERID) %>%
5   group_by(CONTROL) %>%
6   slice(1) %>% # Take first matching row per CONTROL
7   ungroup()
8
9 # Copy donor information to observations with missing data
10 for (i in which(needs_rechomes_fix)) {
11   current_control <- .$CONTROL[i]
12   donor_match <- donor_rows[donor_rows$CONTROL == current_control,
13     ]
14   if (nrow(donor_match) > 0) {
15     rechomes_cols <- names(donor_match)[!names(donor_match) %in% c(
16       "CONTROL")]
17     for (col in rechomes_cols) {
```

```

17   .[[col]][i] <- donor_match[[col]][1]
18 }
19   .has_rechomes_data[i] <- TRUE
20   rows_fixed <- rows_fixed + 1
21 }
22 }

```

This recovery process successfully restored property information for 43 of 66 eligible observations (65.2% recovery rate). The 23 unrecovered observations represent cases where no donor information was available within the test, likely because neither tester received detailed information about the advertised property. After this data recovery, the number of controls with at least one tester having recommendations but missing rechomes data decreased from 143 to 101.

7.4 Final Analytical Dataset Construction

To ensure the integrity of our paired testing analysis, we apply a final filtering step that retains only complete test pairs where both testers have recommended homes data available. This approach is conservative but necessary for analyses that require property-level comparisons between minority and white testers within the same test.

The filtering process requires that observations have at least some recommended homes data and that each CONTROL number includes exactly two testers with such data. This dual requirement ensures that every observation in the final dataset has a valid comparison partner within the same test.

```

1 sales_tester_rechomes <- sales_tester_rechomes %>%
2   filter(has_rechomes_data) %>%
3   group_by(CONTROL) %>%
4   filter(n_distinct(TESTERID) == 2) %>%
5   ungroup()

```

This final filtering step yields our complete analytical dataset of 18,990 property records across 2,714 unique test pairs. The reduction from 3,651 to 2,714 test pairs represents a loss of 937 pairs (25.7%), primarily driven by the requirement that both testers in each pair must have successfully recorded property information. While this attrition is substantial, it ensures that every test in our final sample enables direct within-pair comparisons of the properties shown to minority versus white testers.

The resulting dataset contains several categories of variables for analysis. Tester-level variables include demographics (**TAGE**, **RACE**), test outcomes (**STOTUNIT_TOTAL**, **SAVLBAD_ANY**), and visit characteristics (**first_visit_datetime**, **num_visits**). Property-level variables from the recommended homes data include housing characteristics, location information, and pricing data, all prefixed with “H”. Test-level identifiers include **CONTROL** (linking testers within tests) and **TESTERID** (identifying individual testers).

7.5 Data Quality Implications

The substantial sample reduction during recommended homes integration raises important considerations for analysis interpretation. The final analytical sample represents approximately 73.5% of valid test pairs, suggesting that results should be interpreted as applying to tests where property recommendation data is available rather than the entire population of housing discrimination tests.

However, this sample restriction is methodologically sound for several reasons. First, analyses of discriminatory differences in property recommendations require property data for both testers in each pair. Second, the majority of sample loss occurs randomly due to data collection challenges rather than systematic patterns that might bias results. Third, the remaining sample is sufficiently large (2,684 test pairs) to provide adequate statistical power for detecting meaningful differences in treatment.

8 Superfund Site Matching for Pollution Analysis

To replicate the pollution exposure measures in Christensen and Timmins (2022), we construct counts of Superfund sites within 5 km of each test location. Superfund site locations come from the EPA National Priorities List (NPL) dataset (Excel file `epa-national-priorities-list-ciesin-mod-v2-2014.xls`). We restrict the list to sites with `NPL_STATUS_DATE` prior to 2012 to align with the original study period, and we retain all NPL status categories (Final, Deleted, and Proposed). Distances are computed using the Haversine formula (great-circle distance) with Earth radius set to 6,371 km. The matching is implemented in `superfund_matching.R` and integrated into the data pipeline by adding `SFcount_5km` to the geocoded dataset before the ACS merge.

We validate our construction against the original Christensen and Timmins replication dataset (`recsprocessed_JPE.rds`), which provides their `SFcount` variable and the corresponding test-level coordinates (`Latitude`, `Longitude`). Using their coordinates (which are rounded to two decimal places), our baseline specification matches 92.9% of observations exactly and 99.3% within *pm1*. We also evaluate several alternative specifications that might plausibly differ from the original implementation (e.g., different distance rules or site list cutoffs). These alternatives uniformly reduce exact match rates relative to the baseline, which supports the baseline as the best replication available given the archived data.

All diagnostics reported in the table above are reproduced by `matching_diagnosis.R`, which runs the validation on the full replication dataset and prints the match rates for the baseline and alternative specifications.

Table 1: Superfund Matching Validation Against C&T Replication Data

Specification	Exact match (%)	Within +/-1 (%)
Baseline: Haversine, 5 km, NPL _i 2012	92.9	99.3
Haversine, 5 miles, NPL _i 2012	73.2	88.2
Haversine, 4.9 km, NPL _i 2012	92.8	99.4
Haversine, 5.1 km, NPL _i 2012	92.3	99.3
Haversine, 5.2 km, NPL _i 2012	92.0	99.1
Haversine, 5 km, NPL _i 2011	92.5	99.3
Haversine, 5 km, NPL _i 2013	92.3	99.3
Haversine, 5 km, Final only	87.8	98.5
Haversine, 5 km, Final+Deleted	92.9	99.3
Haversine, 5 km, Final+Proposed	87.9	98.5
Planar circle, 5 km, NPL _i 2012	92.9	99.3
Axis-aligned square, 5 km	88.3	98.2
Manhattan diamond, 5 km	86.8	97.3